

The diagram is ready now.

Floor area estimation from a perspective aisle image

Problem clarification

- **Key constraint:** No physical dimensions are given. A perspective image cannot yield real-world area without scale.
 - **Two valid interpretations:**
 - **Image-plane area** (2D drawing): measurable from the picture's geometry (pixels or relative lengths).
 - **Real-world area** (3D floor): requires at least one real width or length.
-

Image-plane area up to the vanishing point

Assumptions

- **Floor footprint in the image** is a trapezoid: bottom width is nonzero, top width at the vanishing point is effectively zero.
- **Define variables:**
 - $(,w_b,)$ = bottom width of the floor region in the image (distance between shelf bases at the picture's bottom).
 - $(,h_v,)$ = vertical distance in the image from the bottom to the vanishing point along the aisle centerline.

Area of the trapezoid (top width = 0)

$$[A_{\{\text{image, to VP}\}} = \frac{(w_b + 0)^2}{2} \cdot h_v ;=; \frac{w_b \cdot h_v}{2}]$$

Procedure

- **Measure bottom width:** Use the distance between the two floor edge lines at the bottom of the image to get $(,w_b,)$.
 - **Measure height to VP:** From the bottom baseline to the vanishing point gives $(,h_v,)$.
 - **Compute area:** Plug into $(,A = \frac{w_b \cdot h_v}{2},)$.
-

Real-world floor area up to the end of shelves

What is needed

- **At least one real dimension:** Either the **true aisle width** (W) or the **true shelf length** (L).
- Without these, the image alone cannot provide **absolute area**; perspective distorts scale.

If shelves are parallel (constant real width)

$$[A_{\text{real}}] = W \cdot L$$

- **Up to the vanishing point:** In real geometry, the vanishing point represents “far away,” not a physical endpoint. If shelves extend to that point in the image, the **real length** is whatever the store’s shelf length (L) is, not infinite.
- **Up to the end of shelves:** Once (W) and (L) are known (from store specs or measurement), compute ($A = W \cdot L$).

Bridging image to real world (optional, if one scale is known)

If you know one scale (e.g., true aisle width (W))

- **Define pixel width:** Let (w_b^{px}) be the pixel width at the bottom.
- **Scale factor:** ($s = \frac{W}{w_b^{\text{px}}}$) (meters per pixel, locally at the bottom).
- **Local area approximation near the bottom strip:** For a strip of height (Δy) pixels, use ($A \approx s^2 \cdot w_b^{\text{px}} \cdot \Delta y$).
- **Note:** Accurate integration across height requires **projective geometry** or known camera intrinsics to correct perspective foreshortening.

Final expressions

- **Image-plane (up to vanishing point):** $[A_{\text{image, to VP}}] = \frac{w_b \cdot h_v}{2}$
- **Real-world (up to shelf end, shelves parallel):** $[A_{\text{real}}] = W \cdot L$

Without at least one real dimension (W or L), only the image-plane area can be reported.

Definitions

- **Sensor height (S):** The physical vertical size of your camera's image sensor, measured in millimeters. It's the actual hardware dimension of the chip that captures light (e.g., ($S = 4.8 \text{ mm}$)).
 - **Image resolution height (R):** The number of pixels along the vertical axis of the captured image (e.g., ($R = 1920 \text{ px}$)). It's a property of the image file, not the hardware size.
-

How to find them on your phone

- **Sensor height (S):**
 - **From specs:** Look up your phone's camera sensor model (e.g., "1/2.55-inch sensor" translates to a physical height; manufacturers or photography databases list exact mm).
 - **Camera apps:** Some advanced apps show sensor details; otherwise, search the rear camera model online.
 - **Image resolution height (R):**
 - **From the photo:** Check the image's properties; the height in pixels is the resolution height (e.g., 1920 px in a 1920×1080 image).
 - **In-app settings:** Camera settings often list output resolutions (e.g., "1080p," "4K" video; photos might be 3000×4000 px).
-

Why they matter for distance estimation

- **Relating pixels to real size:**
 - With the pinhole camera model, pixel height of an object maps to an angular size using (S) and (R).
 - You can convert focal length in millimeters to "focal length in pixels" via ($f_{\text{px}} = f_{\text{mm}} \cdot \frac{R}{S}$), which is crucial for distance formulas.
- **Device variability:**
 - **Different phones/cameras** have different sensor sizes and focal lengths; using incorrect (S) or (R) leads to wrong distance estimates.
 - **Cropping/zoom modes** change (R) (and sometimes effective (S)), so always use the actual image's resolution and the camera mode's sensor info.

Quick example mapping

- **Given:** ($S = 4.8 \text{ mm}$), ($R = 1920 \text{ px}$), focal length ($f = 4.2 \text{ mm}$).
 - **Focal length in pixels:**
[$f_{\text{px}} = 4.2 \cdot \frac{1920}{4.8} \approx 1680 \text{ px}$]
 - This lets you apply similar-triangle relations between an object's image height (in px) and its real-world distance.
-

Tips for accuracy

- **Use the exact camera:** Rear vs. front sensors differ; match (S) and (f) to the camera used.
- **Check EXIF metadata:** Photos often store focal length and resolution; pair EXIF (R) with known (S).
- **Avoid digital zoom:** It can alter effective parameters; prefer native focal lengths for reliable calculations.

To estimate the **real-world distance** of a 5 cm tall object that appears **5 pixels high** in a phone image, we need to apply **monocular depth scaling** using camera metadata and known object size. Here's how it works:

Step-by-Step Estimation

1. Knowns

- Real object height: ($H_{\text{real}} = 5 \text{ cm}$)
- Image height: ($H_{\text{image}} = 5 \text{ pixels}$)
- Focal length: (f) (in pixels — from camera metadata)

- Sensor height: (S) (in mm — from device specs)

For estimation, we'll assume a typical smartphone camera:

- Focal length: ($f = 4.2, \text{mm}$)
 - Sensor height: ($S = 4.8, \text{mm}$)
 - Image resolution height: ($R = 1920, \text{px}$)
-

2. Convert pixel height to angular size

Using similar triangles: [$\theta = \frac{H_{\text{image}}}{R} \cdot S$]

$$\theta = \frac{5}{1920} \cdot 4.8, \text{mm} = 0.0125, \text{mm}$$

3. Estimate distance using pinhole camera model

$$D = \frac{f \cdot H_{\text{real}}}{H_{\text{image}} \cdot S / R}$$

Plug in values: [$D = \frac{4.2, \text{mm} \cdot 50, \text{mm}}{(5, \text{px} \cdot 4.8, \text{mm}) / 1920} = \frac{210}{0.0125} = 16,800, \text{mm} = 16.8, \text{meters}$]

Final Answer

Estimated real-world distance: ~16.8 meters

Assumptions

- Object is upright and perpendicular to camera.
 - No lens distortion or zoom.
 - Focal length and sensor size are typical for smartphones.
 - Lighting and resolution are sufficient for clear detection.
-

Would you like me to generate a **visual diagram** showing how this triangle-based estimation works from image pixels to real-world distance?

Perfect — let's go step by step and show **how each method works in practice**. These are the three main ways to recover *absolute* distance without knowing the object's height.

1. Known Baseline (Structure from Motion)

- **Setup:** Take two photos of the same scene, moving the phone sideways by a known baseline (b) (say 10 cm).
 - **Principle:** Nearby objects shift more in the image than distant ones (disparity).
 - **Formula:** $[Z = \frac{f_{\text{px}} \cdot b}{d}]$
 - (Z) = distance to object
 - (f_{px}) = focal length in pixels
 - (b) = baseline translation (meters)
 - (d) = disparity (pixel shift of the object between the two frames)
 - **Example:** If you move the phone 0.1 m sideways and the object shifts 20 px, with ($f_{\text{px}} = 1680$), then: $[Z = \frac{1680 \cdot 0.1}{20} = 8.4, \text{m}]$
-

2. Fiducial Marker / Known Reference

- **Setup:** Place something of known size in the scene (credit card = 8.5 cm × 5.4 cm, A4 paper = 29.7 cm × 21 cm).
 - **Principle:** Use the known dimension as a scale anchor.
 - **Formula:** $[D = \frac{f_{\text{px}} \cdot H_{\text{real}}}{H_{\text{image}}}]$
 - Same as the pinhole model, but now (H_{real}) is the fiducial's known size.
 - **Example:** If an A4 paper (29.7 cm tall) appears 300 px tall in the image, with ($f_{\text{px}} = 1680$): $[D = \frac{1680 \cdot 297}{300} \approx 1663, \text{mm}]$; $[D = 1.66, \text{m}]$
-

3. Camera Height & Ground Plane

- **Setup:** If the object touches the floor, measure your phone's lens height above ground (h_c).
- **Principle:** Geometry of the ground plane + camera tilt gives distance.
- **Formula (simplified for level camera):** $[Z = \frac{h_c \cdot f_{\text{px}}}{y - y_0}]$
 - (Z) = distance to ground point
 - (h_c) = camera height above ground
 - (f_{px}) = focal length in pixels
 - (y) = pixel row of the object's base

- (y_0) = principal point (center row)
 - **Example:** If camera height = 1.5 m, $(f_{\text{px}}) = 1680$, and the base of the object is 400 px below the image center: $[Z = \frac{1.5 \cdot 1680}{400} = 6.3, \text{m}]$
-

✓ Takeaway

- **Baseline sweep:** Use motion and disparity.
- **Fiducial marker:** Use a known object size as a scale.
- **Camera height:** Use geometry of the ground plane.

All three methods give you **absolute distance** without needing the target's height.

Would you like me to **draw a comparative diagram** showing these three setups side by side (baseline shift, fiducial marker, and ground-plane geometry)? That way you can visually see how each method works.

Using a pre-taken picture as a fiducial on screen

You can do this, but it only works if the pre-taken image is displayed on a physical screen that is placed in the same scene as the object. Simply showing a side-by-side montage on your phone screen (reference next to the object's photo) does not provide any metric scale because the two images don't share the same projective geometry. The on-screen reference must be physically in the camera's field of view.

When it works

- **Digital fiducial on a screen in the scene:** Display a known-size marker on a second device (or your own phone), place that screen next to the target object, then photograph both together. The screen acts like a printed fiducial if you know its physical pixel pitch.

- **Same plane and orientation:** The screen face should be in the same plane as the object, and roughly parallel to the camera to minimize perspective distortion.
 - **Known screen PPI (pixels per inch):** Convert device pixels to millimeters so the displayed marker has a known physical size.
-

How to set it up

- **Determine pixel pitch:**
 - **Label:** PPI
 - **Formula:**

$$[\text{mm/px}] =; \frac{25.4}{\text{PPI}}]$$
 - **Design the marker size in device pixels:**
 - **Label:** Desired physical size (H_{real}) (mm)
 - **Formula:**

$$[H_{\text{px}}] =; \frac{H_{\text{real}}}{\text{mm/px}} =; H_{\text{real}} \cdot \frac{\text{PPI}}{25.4}]$$
 - **Render at native scale:**
 - **Label:** Ensure 1:1 device pixels
 - **Tip:** Use full-screen, disable zoom/scaling, avoid “Display size” accessibility scaling, and account for safe areas/notches.
 - **Place and capture:**
 - **Label:** Scene placement
 - **Tip:** Put the screen right next to the object, same distance from the camera, and shoot head-on to reduce skew.
-

Computing distance with the on-screen fiducial

- **Measure the fiducial in the photo:**
 - **Label:** (H_{image}) = pixel height of the on-screen marker in the captured photo.
- **Use the pinhole model for distance to the marker (and the object beside it):**

$$[D] =; \frac{f_{\text{px}}}{H_{\text{real}}} \cdot H_{\text{image}}]$$
 - **Label:** (f_{px})
 - **Formula:**

$$[f_{\text{px}}] =; f_{\text{mm}} \cdot \frac{R}{S}]$$
 - **Labels:** (H_{real}) = physical height of the displayed marker, (R) = image resolution height (px), (S) = sensor height (mm).

- **Or use the fiducial purely as a scale for nearby objects:**
 - **Label:** Scale factor
 - **Formula:**

$$\left[s \stackrel{?}{=} \frac{H_{\text{real}}}{H_{\text{image}}} \quad \rightarrow \quad T_{\text{real}} \stackrel{?}{=} s \cdot T_{\text{image}} \right]$$
 - **Note:** Valid if the object is in the same plane as the fiducial.
-

Common pitfalls and fixes

- **Screen scaling and DPI mismatches:**
 - **Fix:** Verify with a physical ruler; adjust the pixel size until the displayed marker measures correctly in mm.
 - **Perspective skew (not parallel to camera):**
 - **Fix:** Align the screen and object faces; use the central region to measure.
 - **Reflections, glare, PWM flicker, rolling shutter:**
 - **Fix:** Lower brightness, use matte screen or viewing angle, increase exposure time.
 - **Different planes (object behind/in front):**
 - **Fix:** Place the screen and object at the same depth from the camera, or switch to the pinhole distance formula per item.
-

What does not work

- **Side-by-side images on the phone UI:**
 - **Reason:** No shared geometry; you cannot infer metric scale between separate images.
 - **Unknown PPI or scaled rendering:**
 - **Reason:** You don't know the real physical size of the on-screen marker.
-

Quick verdict

- **Yes, it's possible** if the pre-taken picture is displayed as a calibrated on-screen fiducial that appears in the same physical photo as the object.
- **No, it's not sufficient** if the images are only juxtaposed digitally; they must share the same captured scene to enable metric estimation.

If you share your device model (for PPI) and the marker size you want to display, I can give you the exact pixel dimensions to render and a short checklist to validate the setup.

Great question — you're essentially asking how a **software-based baseline sweep (structure-from-motion)** stacks up against **hardware LiDAR sensors** when implemented as a feature on smartphones. Let's break it down across the dimensions you listed:

Heat & Energy Consumption

- **Baseline sweep (software):**
 - Relies on the phone's existing camera and CPU/GPU for image capture and disparity calculation.
 - Energy cost comes from image processing (optical flow, feature matching).
 - Heat is moderate, mostly tied to processor load during computation.
- **LiDAR (hardware):**
 - Uses an active infrared laser emitter and sensor.
 - Continuous scanning consumes more power than a single photo pair.
 - Generates more heat if used for extended AR sessions.

👉 **Winner:** Baseline sweep is lighter on energy, since it piggybacks on the camera.

Speed

- **Baseline sweep:**
 - Requires at least two frames and motion (user must move phone).
 - Processing time depends on algorithm efficiency (can be real-time with optimized code).
- **LiDAR:**
 - Instant depth map from a single scan.
 - Real-time performance, no user motion required.

👉 **Winner:** LiDAR is faster and more seamless.

Accuracy

- **Baseline sweep:**
 - Accuracy depends on baseline size, camera calibration, and feature quality.

- Small baselines (like 5–10 cm phone shifts) limit precision at long ranges.
- Sensitive to textureless surfaces (white walls, shiny objects).
- **LiDAR:**
 - Provides direct depth measurements with centimeter-level accuracy.
 - Works well even in low-texture scenes, but can struggle with transparent or reflective surfaces.

👉 **Winner:** LiDAR, especially for absolute metric accuracy.

Efficiency

- **Baseline sweep:**
 - Efficient for relative depth and scale if fiducials or known baselines are used.
 - No extra hardware needed.
- **LiDAR:**
 - Efficient for absolute depth maps, but requires specialized hardware.

👉 **Winner:** Depends on context — baseline sweep is efficient in software-only scenarios, LiDAR is efficient for robust AR.

Size & Hardware Footprint

- **Baseline sweep:**
 - No extra hardware; uses existing camera.
 - Works on any phone with a decent camera and processor.
- **LiDAR:**
 - Requires dedicated sensor module (adds bulk, cost, and design constraints).
 - Only available on select high-end devices (e.g., iPhone Pro models, some Android flagships).

👉 **Winner:** Baseline sweep — zero hardware footprint.

Processor Strain

- **Baseline sweep:**
 - Heavy image processing (feature detection, matching, disparity maps).
 - Can strain CPU/GPU, especially on mid-range phones.

- **LiDAR:**
 - Offloads depth sensing to dedicated hardware.
 - Minimal processor strain; results delivered as ready-made depth maps.

👉 **Winner:** LiDAR — less strain on the main processor.

Flexibility Across Phones

- **Baseline sweep:**
 - Universally usable — any phone with a camera can run it.
 - Performance scales with processor power.
- **LiDAR:**
 - Limited to phones with built-in LiDAR modules.
 - Not portable across most devices.

👉 **Winner:** Baseline sweep — far more flexible and deployable.

Summary Comparison

Dimension	Baseline Sweep (Software)	LiDAR (Hardware)
Heat/Energy	Lower	Higher
Speed	Slower (needs motion)	Instant
Accuracy	Moderate, baseline-limited	High, direct
Efficiency	Good for relative depth	Excellent for absolute depth
Size	No extra hardware	Needs sensor
Processor Strain	Higher (CPU/GPU load)	Lower (dedicated)
Flexibility	Works on all phones	Only select models

Takeaway

- **Baseline sweep:** Best for universal deployment, low-cost apps, and accessibility features where hardware isn't guaranteed.
- **LiDAR:** Superior for speed and accuracy, but limited to premium devices with dedicated sensors.

Michael, given your focus on **accessible tech for visually impaired users**, a baseline sweep implementation could be more inclusive — it would run on almost any phone, even budget models, whereas LiDAR is locked to high-end devices.

Would you like me to sketch out a **prototype algorithm flow** (camera capture → disparity calculation → distance estimation) so you can see how baseline sweep could be coded into a mobile app?

Rescale tap coordinates to the original image size.

- If you downsample to display, compute a scale factor and map taps back:

$x_{orig} = x_{tap} \cdot W_{orig} / W_{display}$, $y_{orig} = y_{tap} \cdot H_{orig} / H_{display}$

Then use disparity in the original pixel space.

Use the actual focal length in mm, not the 35 mm equivalent.

- Prefer `photo.exif.FocalLength` (mm). Avoid `FocalLengthIn35mmFilm`.

Compute fpx with matching axis.

- If disparity is horizontal, use sensor width (mm) and image width (px). For vertical disparity, use the height pair:

$fpx = fmm \cdot W_{orig} / \text{sensor width (mm)}$

Rectify your pair (as best you can).

- Keep the phone level and translate sideways along a ruler. No rotation.

Average multiple corresponding points.

- Already in place — keep it, but rescaled.